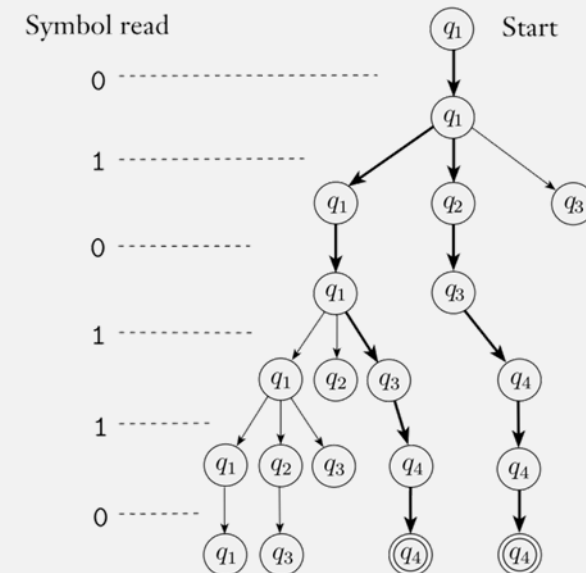


# CS 420

## Computing with NFAs

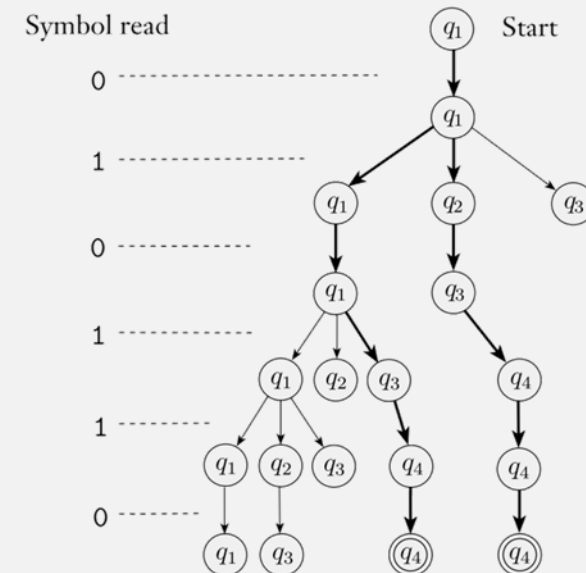
Tuesday, October 6, 2026

# UMass Boston Computer Science



# Announcements

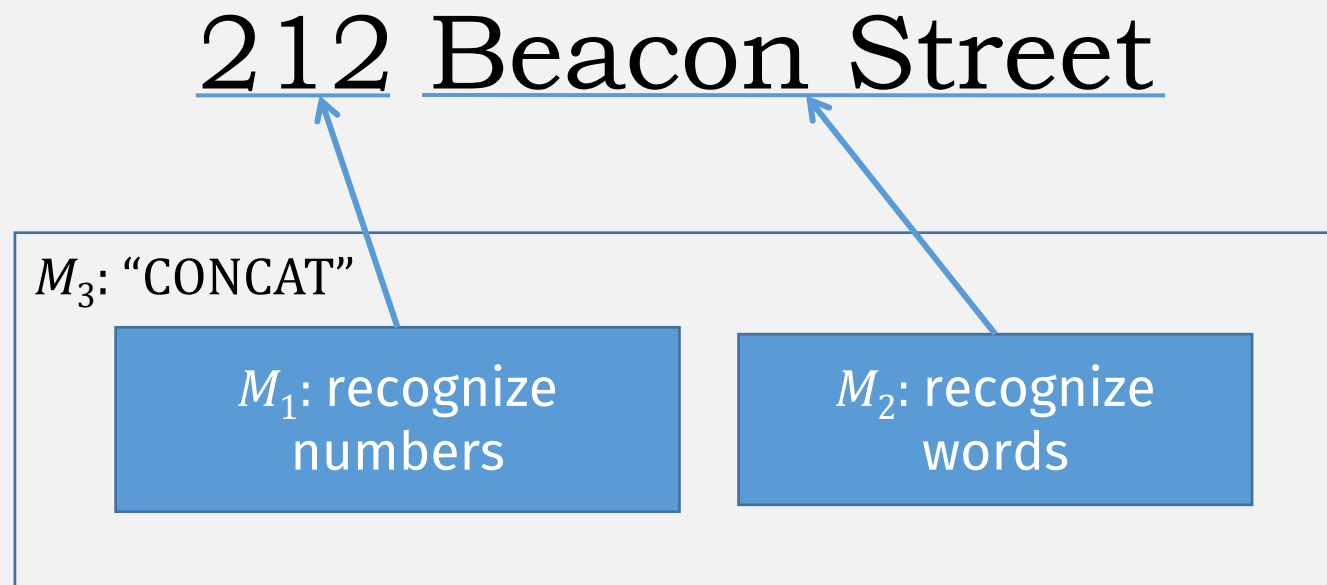
- HW 2 grades out
    - Questions / complaints must go through GradeScope request
    - Do not ask me or TA directly (cannot resolve in lecture)
  - Hints:
    - **Proofs** (and other mathematical definitions) are **precise**, and **self-explanatory!**
    - **Precise answers demonstrate understanding ... imprecise the opposite**
    - **Incorrect answers cannot be fixed with extraneous comments, multiple solutions!** (not necessary to teach the course staff)
  - HW 3 in
    - ~~Due: Tue 10/6~~
  - HW 4 out
    - Out: Tue 10/6
    - Out: Tue 10/13 12:30pm
- 
- Symbol read
- 0 -----
- 1 -----
- 0 -----
- 1 -----
- 1 -----
- q<sub>1</sub>
- q<sub>1</sub>
- q<sub>1</sub> q<sub>2</sub> q<sub>3</sub>
- q<sub>1</sub> q<sub>2</sub> q<sub>3</sub>



*Previously*

## Another (common string) operation: Concatenation

Example: Recognizing street addresses



**Concatenation:**  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

(infix function notation)

# Concatenation of Languages

Let the alphabet  $\Sigma$  be the standard 26 letters  $\{a, b, \dots, z\}$ .

If  $A = \{\text{fort}, \text{south}\}$   $B = \{\text{point}, \text{boston}\}$

$$A \circ B = \{\text{fortpoint}, \text{fortboston}, \text{southpoint}, \text{southboston}\}$$

# Regular Languages Closed Under Concat?

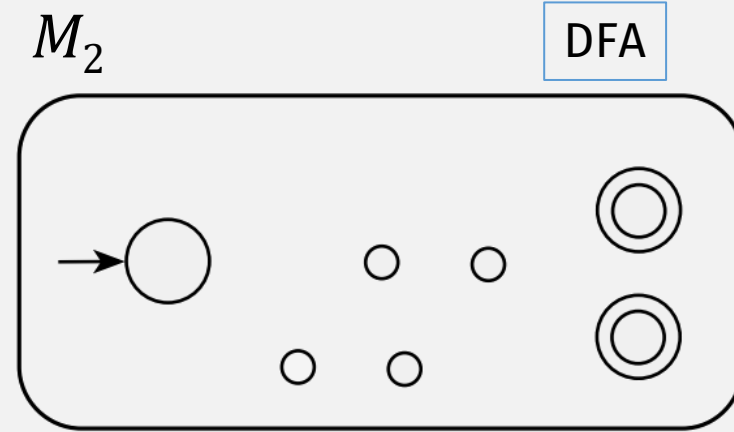
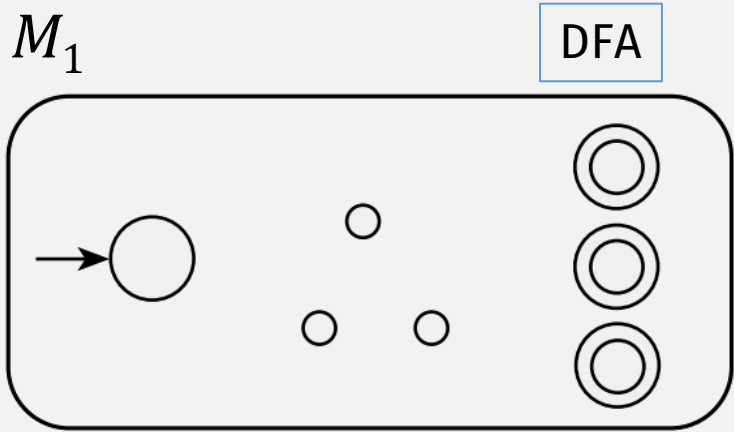
## THEOREM .....

The class of regular languages is closed under the concatenation operation.

In other words, IF  $A_1$  IS-A **Regular** Language AND  $A_2$  IS-A **Regular** Language  
THEN  $A_1 \circ A_2$  IS-A **Regular** Language

- Construct a new machine  $M$  recognizing  $A_1 \circ A_2$ ? (like union)
  - Using DFA  $M_1$  (which recognizes  $A_1$ ),
  - and DFA  $M_2$  (which recognizes  $A_2$ )

Concatentation

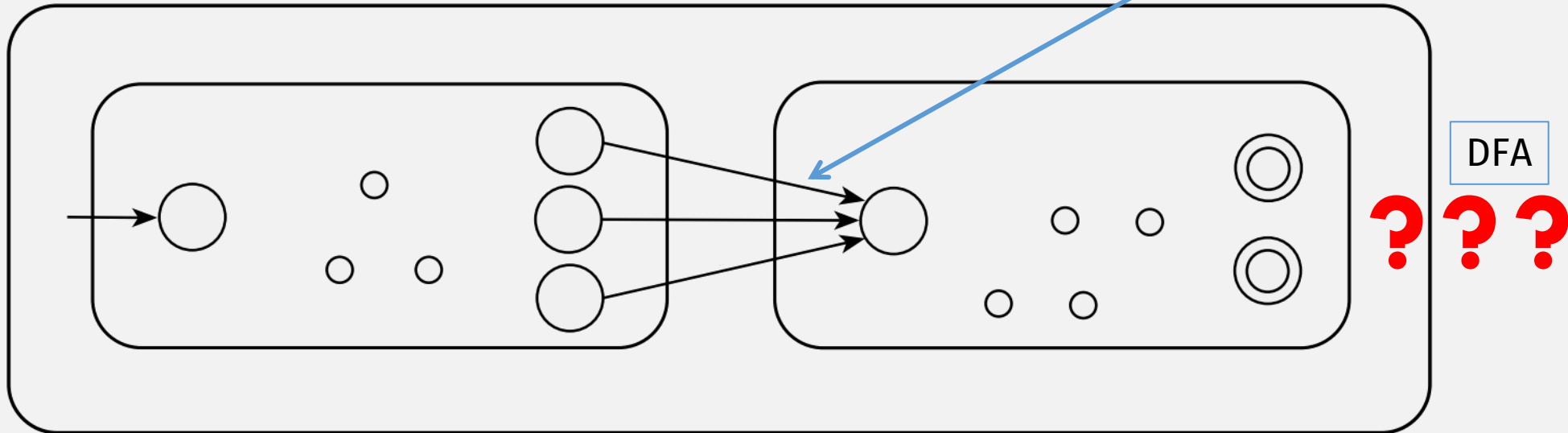


PROBLEM:  
Can only read input once, can't backtrack

Have:  $M_1$  recognize  $A_1$        $M_2$  recognize  $A_2$ .

Want: Construction of  $M$  to recognize  $A_1 \circ A_2$

Need to switch machines at some point, but when?



**Concatenation:**  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

# Overlapping Concatenation Example

- Let  $M_1$  ... recognize language  $A = \{ \text{j en, j ens} \}$
- and  $M_2$  ... recognize language  $B = \{ \text{s mith} \}$
- Want: Construct  $M$  to recognize  $A \circ B = \{ \text{j en s mith, j ens mith} \}$
- If  $M$  reads **j en** ... and next char is **s**
- $M$  must decide to either:

**Concatenation:**  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

# Overlapping Concatenation Example

- Let  $M_1$  ... recognize language  $A = \{ \text{j en, j ens} \}$
- and  $M_2$  ... recognize language  $B = \{ \text{s mith} \}$
- Want: Construct  $M$  to recognize  $A \circ B = \{ \text{j ensmith, j ens} \text{s mith} \}$
- If  $M$  reads **j en** ... and next char is **s**
- $M$  must decide to either:
  - stay in  $M_1$  (correct, if full input is **j en****s****mith**)



**Concatenation:**  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

# Overlapping Concatenation Example

- Let  $M_1$  ... recognize language  $A = \{ \text{j en, j ens} \}$
- and  $M_2$  ... recognize language  $B = \{ \text{smith} \}$
- Want: Construct  $M$  to recognize  $A \circ B = \{ \text{j ens} \text{smith, j enssmith} \}$
- If  $M$  reads **j en** ... and next char is **s**
- $M$  must decide to either:
  - stay in  $M_1$  (correct, if full input is **j enssmith**)
  - or switch to  $M_2$  (correct, if full input is **j ens****smith**)
- But to recognize  $A \circ B$ , it needs to handle both cases!!
  - Without backtracking

**A DFA can't do this!**

(seemingly)

# Regular Languages Closed Under Concat?

**FALSE?**

## THEOREM .....

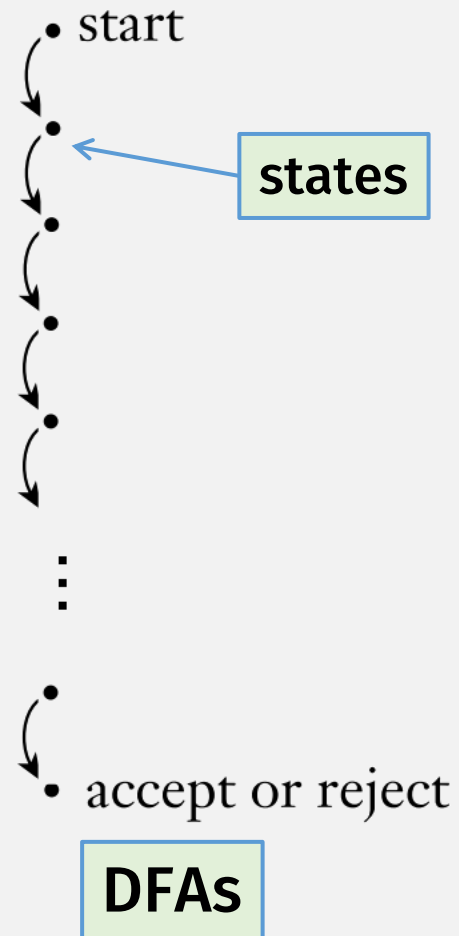
The class of regular languages is closed under the concatenation operation.

- Cannot combine  $A_1$  and  $A_2$ 's machine because:
  - Need to switch from  $A_1$  to  $A_2$  at some point ...
  - ... but we don't know when! (we can only read input once)
- This requires a new kind of machine!
- But does this mean concatenation is not closed for regular langs?

# **Nondeterminism**

# Deterministic vs Nondeterministic

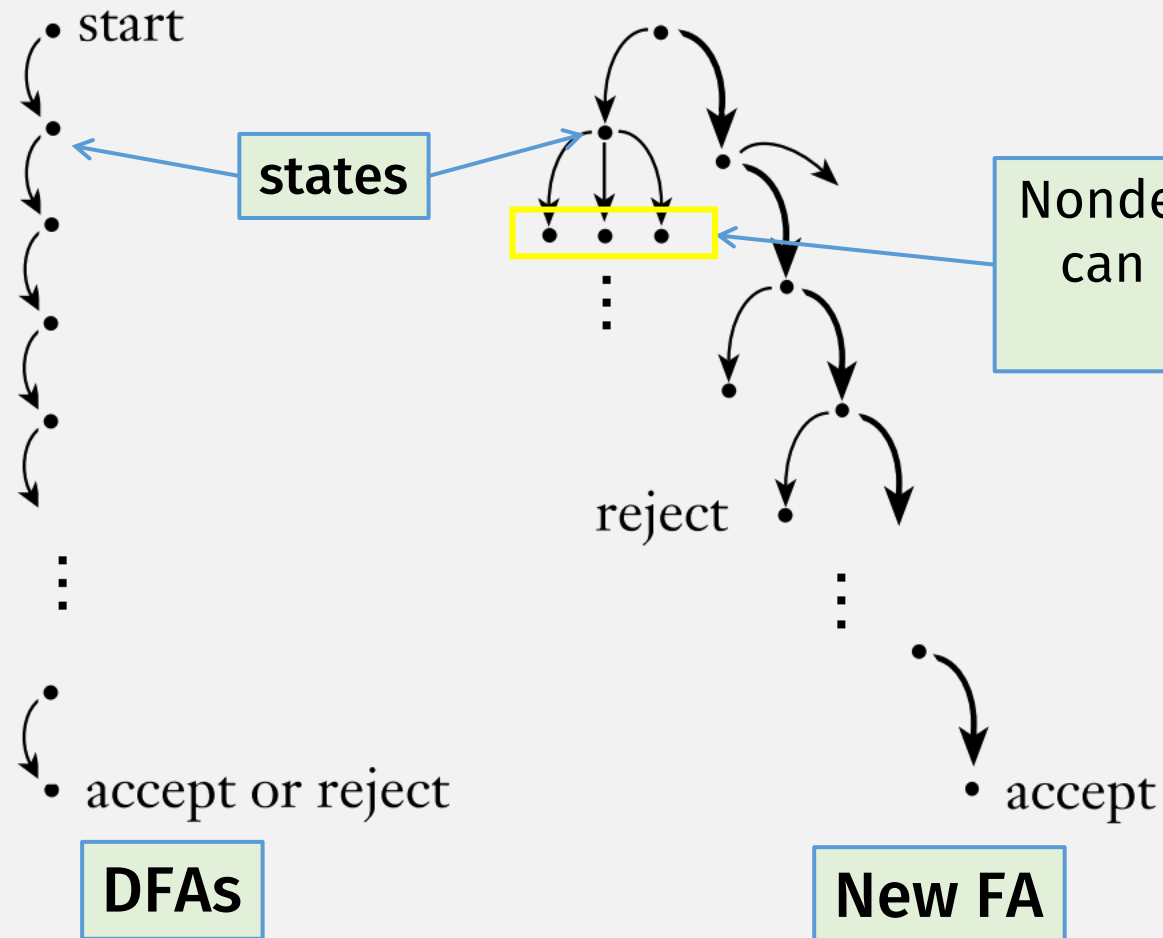
Deterministic  
computation



# Deterministic vs Nondeterministic

Deterministic  
computation

Nondeterministic  
computation



# DFA: The Formal Definition

## DEFINITION

deterministic

A *finite automaton* is a  $\text{mk-DFA}$   $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set called the *states*,
2.  $\Sigma$  is a finite set called the *alphabet*,
3.  $\delta: Q \times \Sigma \longrightarrow Q$  is the *transition function*,
4.  $q_0 \in Q$  is the *start state*, and
5.  $F \subseteq Q$  is the *set of accept states*.

**Deterministic Finite Automata (DFA)**

# Nondeterministic Finite Automata (NFA)

DefNFA

## DEFINITION

Compare with DFA:

A *finite automaton* is a mk-DFA  $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set called the *states*,
2.  $\Sigma$  is a finite set called the *alphabet*,
3.  $\delta: Q \times \Sigma \rightarrow Q$  is the *transition function*,
4.  $q_0 \in Q$  is the *start state*, and
5.  $F \subseteq Q$  is the *set of accept states*.

A *nondeterministic finite automaton* is a mk-NFA  $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set of states,
2.  $\Sigma$  is a finite alphabet,
3.  $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$  is the transition function,
4.  $q_0 \in Q$  is the start state, and
5.  $F \subseteq Q$  is the set of accept states.

Difference

Power set, i.e. a transition results in set of states

# Power Sets

- A **power set** is the set of all subsets of a set
- Example:  $S = \{a, b, c\}$
- Power set of  $S =$ 
  - $\{ \{ \}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} \}$
  - Note: includes the empty set!



# Nondeterministic Finite Automata (NFA)

## DEFINITION

---

A *nondeterministic finite automaton*

is a **mk-NFA**  $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set of states,
2.  $\Sigma$  is a finite alphabet,
3.  $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function,
4.  $q_0 \in Q$  is the start state, and
5.  $F \subseteq Q$  is the set of accept states.

Transition label can be “empty”,  
i.e., machine can transition  
without reading input

$$\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$$

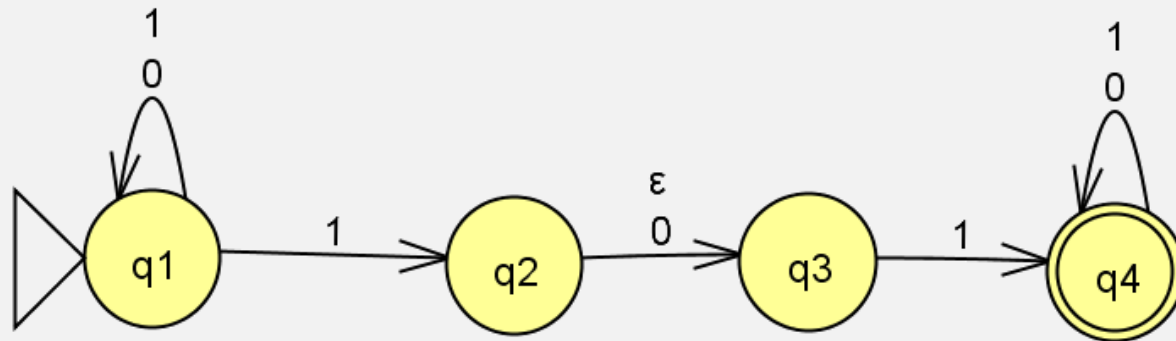
CAREFUL:

$\epsilon$  symbol is reused here, as a transition label  
(ie, an argument to  $\delta$ )

- **It's not the empty string!**
- And it's (still) not a character in the alphabet  $\Sigma$ !

# NFA Example

- Come up with a formal description of the following NFA:



## DEFINITION

A *nondeterministic finite automaton*

is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set of states,
2.  $\Sigma$  is a finite alphabet,
3.  $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function,
4.  $q_0 \in Q$  is the start state, and
5.  $F \subseteq Q$  is the set of accept states.

let  $N_1 = \text{mk-NFA}( Q, \Sigma, \delta, q_1, F )$ , where

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0,1\}$$

$$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$$

Empty transition  
(no input read)

Result of transition  
is a set

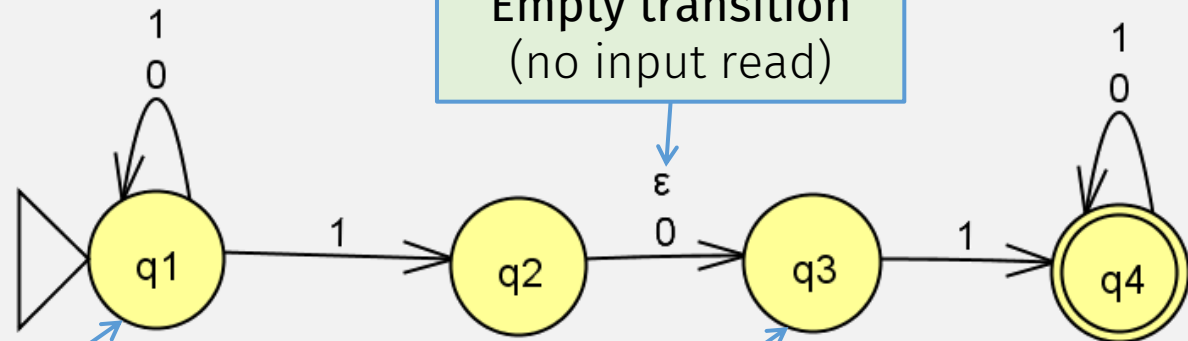
|       | 0           | 1              | $\epsilon$  |
|-------|-------------|----------------|-------------|
| $q_1$ | $\{q_1\}$   | $\{q_1, q_2\}$ | $\emptyset$ |
| $q_2$ | $\{q_3\}$   | $\emptyset$    | $\{q_3\}$   |
| $q_3$ | $\emptyset$ | $\{q_4\}$      | $\emptyset$ |
| $q_4$ | $\{q_4\}$   | $\{q_4\}$      | $\emptyset$ |

$$F = \{q_4\}$$

Empty transition  
(no input read)

Multiple 1 transitions

No 0 transition



# Another NFA

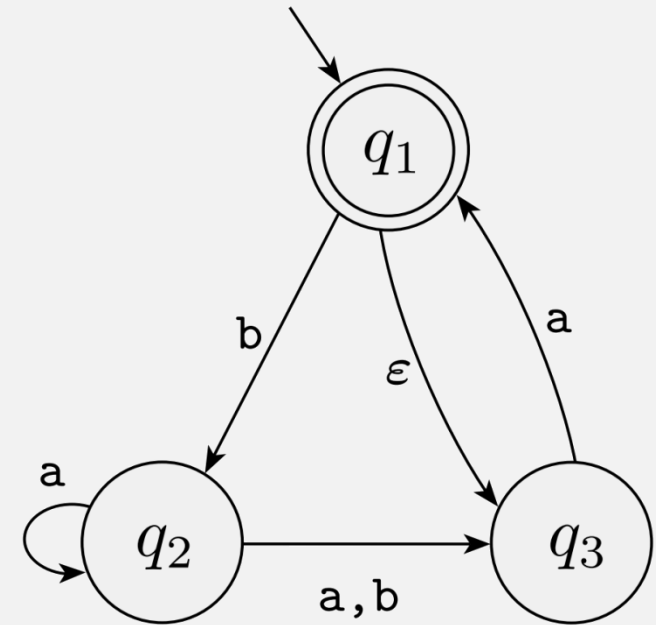
$$\Sigma = \{ \mathbf{a}, \mathbf{b} \}$$

## DEFINITION

A *nondeterministic finite automaton*

is a **mk-NFA** $(Q, \Sigma, \delta, q_0, F)$ , where

1.  $Q$  is a finite set of states,
2.  $\Sigma$  is a finite alphabet,
3.  $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function,
4.  $q_0 \in Q$  is the start state, and
5.  $F \subseteq Q$  is the set of accept states.



# In-class Exercise Solution

let  $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F)$ , where

$$Q = \{q_1, q_2, q_3\}$$

$$\Sigma = \{\mathbf{a}, \mathbf{b}\}$$

$$\delta: Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q) \rightarrow$$

$$F = \{q_1\}$$

$$\delta(q_1, \mathbf{a}) = \{\}$$

$$\delta(q_1, \mathbf{b}) = \{q_2\}$$

$$\delta(q_1, \varepsilon) = \{q_3\}$$

$$\delta(q_2, \mathbf{a}) = \{q_2, q_3\}$$

$$\delta(q_2, \mathbf{b}) = \{q_3\}$$

$$\delta(q_2, \varepsilon) = \{\}$$

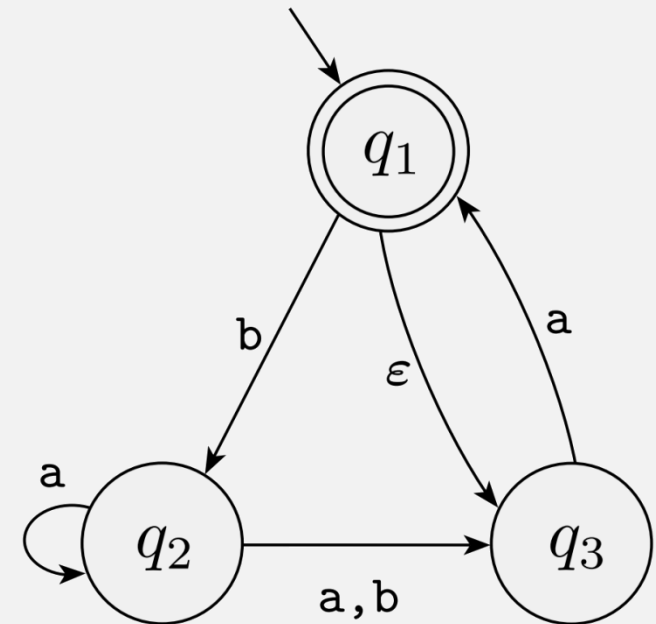
$$\delta(q_3, \mathbf{a}) = \{q_1\}$$

$$\delta(q_3, \mathbf{b}) = \{\}$$

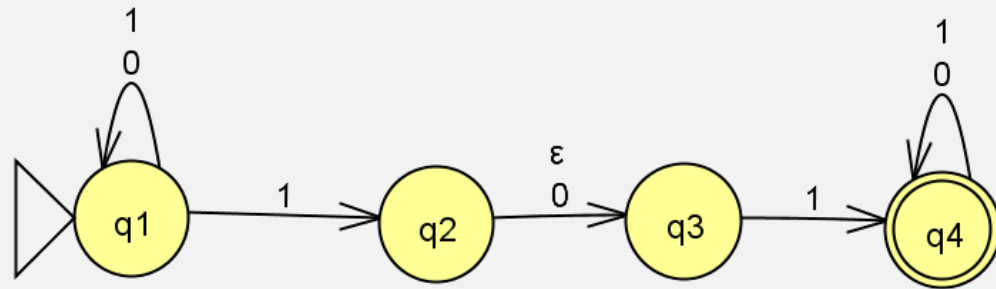
$$\delta(q_3, \varepsilon) = \{\}$$

Differences with DFA?

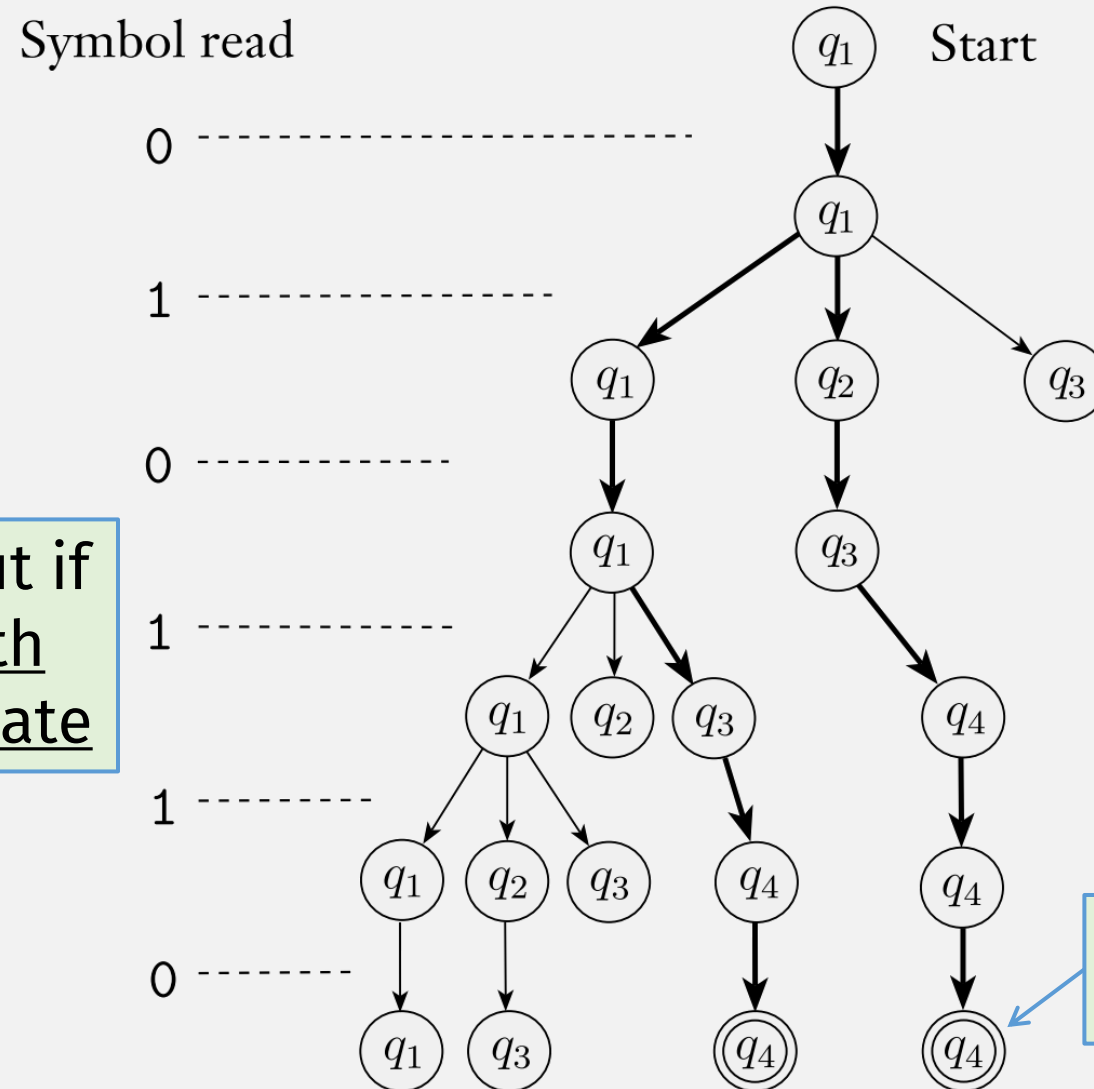
- $\delta$  output is a set
- state doesn't need transition for every alphabet symbol
- state can have multiple transitions for one symbol
- can have "empty" transitions ( $\delta$  output is empty set)



# NFA Computation (JFLAP demo): 010110



# NFA Computation Sequence



NFA **accepts** input if  
at least one path  
ends in accept state

Each step can  
branch into  
multiple states at  
the same time!

So this is an **accepting computation**

# DFA Computation Rules

## Informally

Given

- A **DFA** (~ a “Program”)
- and **Input** = **string of chars**, e.g. “1101”

A DFA computation (~ “Program run”):

- Start in *start state*
- Repeat:
  - Read 1 char from **Input**, and
  - Change state according to *transition rules*

Result of computation:

- Accept if last state is *Accept state*
- Reject otherwise

## Formally (i.e., mathematically)

- $M = \text{mk-DFA}( Q, \Sigma, \delta, q_0, F ) \dots$
- $w = w_1 w_2 \cdots w_n$

A DFA **computation** is a sequence of states:

- specified by  $\hat{\delta}(q_0, w)$  where:

- $M$  **accepts**  $w$  if  $\hat{\delta}(q_0, w) \in F$
- $M$  **rejects** otherwise



# DFA Computation Rules

## *Informally*

Given

- A **DFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

A **DFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
  - Read 1 char from **Input**, and
  - Change state according to *transition rules*

Result of computation:

- Accept if last state is *Accept state*
- Reject otherwise

## *Formally (i.e., mathematically)*

- $M = \text{mk-DFA}( Q, \Sigma, \delta, q_0, F ) \dots$
- $w = w_1 w_2 \cdots w_n$

A **DFA computation** is a sequence of states:

- specified by  $\hat{\delta}(q_0, w)$  where:

- $M$  **accepts**  $w$  if  $\hat{\delta}(q_0, w) \in F$
- $M$  **rejects** otherwise

Ignoring  $\epsilon$  transitions, for now!

# NFA Computation Rules

## Informally

Given

- An **NFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

An **NFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
  - Read 1 char from **Input**, and

For each “current” state, according to *transition rules*  
go to next states

... then combine all “next states”

Result of computation:

- Accept if last **set of states has accept state**
- Reject otherwise

## Formally (i.e., mathematically)

- $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F) \dots$
- $w = w_1 w_2 \cdots w_n$

An **NFA computation** is a ...

- specified by  $\hat{\delta}(q_0, w)$  where:

- $M$  **accepts**  $w$  if ...
- $M$  **rejects** ...

Ignoring  $\epsilon$  transitions, for now!

# NFA Computation Rules

## Informally

Given

- An **NFA** (~ a “Program”)
- and **Input** = string of chars, e.g. “1101”

An **NFA computation** (~ “Program run”):

- Start in *start state*
- Repeat:
  - Read 1 char from **Input**, and

For each “current” state, according to *transition rules*  
go to next states

... then combine all “next states”

Result of computation:

- Accept if last **set of states has accept state**
- Reject otherwise

## Formally (i.e., mathematically)

- $N = \text{mk-NFA}( Q, \Sigma, \delta, q_0, F ) \dots$
- $w = w_1 w_2 \cdots w_n$

An **NFA computation** is a sequence of:  
sets of states

e.g.,  $\{q_0\}, \{q_i, \dots\}, \dots$

- specified by  $\hat{\delta}(q_0, w)$  where:

???

- $M$  **accepts**  $w$  if ...
- $M$  **rejects** ...

# DFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - state  $q \in Q$  (doesn't have to be an accept state)

Recursive Input Data  
needs  
Recursive Function

Base case

Base case

$$\hat{\delta}(q, \epsilon) =$$

A **String** is either:

- the **empty string** ( $\epsilon$ ), or
- $xa$  (non-empty string) where
  - $x$  is a **string**
  - $a$  is a "char" in  $\Sigma$

# DFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - state  $q \in Q$  (doesn't have to be an accept state)

(Defined recursively)

Base case  $\hat{\delta}(q, \varepsilon) = q$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \delta(\hat{\delta}(q, w'), w_n)$$

where  $w' = w_1 \cdots w_{n-1}$

Recursive Input Data  
needs  
Recursive Function

A **String** is either:

- the **empty string** ( $\varepsilon$ ), or
- $xa$  (non-empty string) where

- $x$  is a **string**
- $a$  is a "char" in  $\Sigma$

Recursion  
on string

string

char

Recursive case

"smaller" argument

"second to last" state

# DFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow Q$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - state  $q \in Q$  (doesn't have to be an accept state)

(Defined recursively)

Base case  $\hat{\delta}(q, \varepsilon) = q$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \delta(\hat{\delta}(q, w'), w_n)$$

where  $w' = w_1 \cdots w_{n-1}$

Recursive Input Data  
needs  
Recursive Function

A **String** is either:

- the **empty string** ( $\varepsilon$ ), or
- $xa$  (non-empty string)  
where
  - $x$  is a **string**
  - $a$  is a “char” in  $\Sigma$

Single step from “second to last” state  
and last char gets to last state

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function

## NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$

- Range (output):

states

$$qs \subseteq Q$$

Result is set of states

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function

# NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - states  $qs \subseteq Q$

Result is set of states

(Defined recursively)

Base case

$$\hat{\delta}(q, \epsilon) = \{q\}$$

Recursively Defined Input  
needs  
Recursive Function

Base case

A **String** is either:

- the **empty string** ( $\epsilon$ ), or
- $xa$  (non-empty string) where
  - $x$  is a **string**
  - $a$  is a "char" in  $\Sigma$



$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function

# NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - states  $qs \subseteq Q$

(Defined recursively)

Base case  $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive Case

$$\hat{\delta}(q, w'w_n) =$$

where  $w' = w_1 \cdots w_{n-1}$

Recursive case

Recursively Defined Input  
needs  
Recursive Function

A **String** is either:

- the **empty string** ( $\epsilon$ ), or
- $xa$  (non-empty string) where
  - $x$  is a **string**
  - $a$  is a "char" in  $\Sigma$

Recursive part

Recursion on recursive part

"second to last"  
set of states

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function

# NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - states  $qs \subseteq Q$

(Defined recursively)

Base case  $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{i=1}^k \delta(q_i, w_n)$$

where  $w' = w_1 \cdots w_{n-1}$

For each "second to last" state, take single step on last char

Recursively Defined Input needs Recursive Function

A **String** is either:

- the **empty string** ( $\epsilon$ ), or
- $xa$  (non-empty string) where
  - $x$  is a **string**
  - $a$  is a "char" in  $\Sigma$

last char

"second to last" set of states

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  is the transition function

# NFA Multi-Step Transition Function

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (input)
  - state  $q \in Q$
  - string  $w = w_1 \cdots w_n$
- Range (output)
  - set of states  $qs \subseteq Q$

Given

- An NFA (~ a “Program”)
- and Input = string of chars, e.g. “1101”

A DFA computation (~ “Program run”):

- Start in *start state*

- Repeat:

- Read 1 char from Input, and

according to *transition rules*

For each “current” state,  
go to next states

... then combine all sets of “next states”

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{i=1}^k \delta(q_i, w_n)$$

where  $w' = w_1 \cdots w_{n-1}$

Still ignoring  $\epsilon$  transitions!

Recursively Defined Input  
needs

- the **empty string** ( $\epsilon$ ), or
- $xa$  (non-empty string)  
where
  - $x$  is a **string**
  - $a$  is a “char” in  $\Sigma$

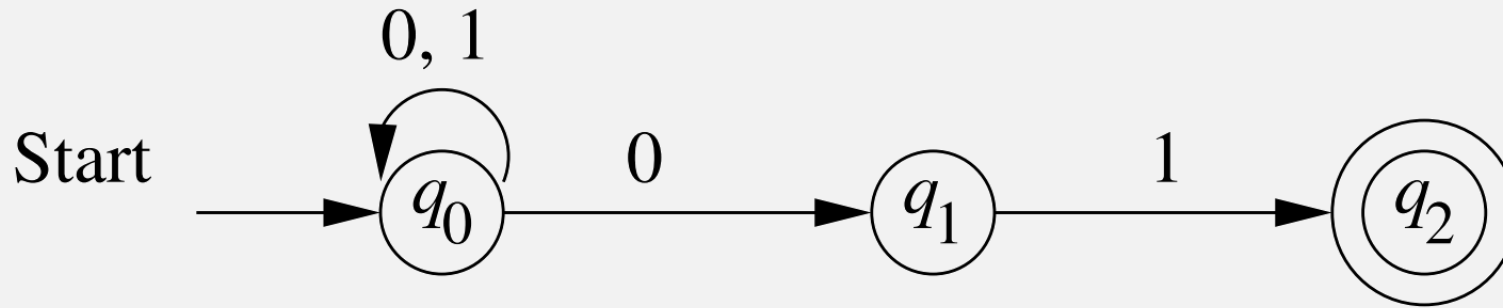
$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

Base case:  $\hat{\delta}(q, \epsilon) = \{q\}$

Recursive case:  $\hat{\delta}(q, w) = \bigcup_{i=1}^k \delta(q_i, w_n)$   
where:

$\hat{\delta}(q, w_1 \cdots w_{n-1}) = \{q_1, \dots, q_k\}$

# NFA Multi-Step $\delta$ Example



- $\hat{\delta}(q_0, \epsilon) = \{q_0\}$

We haven't considered  
**empty transitions!**

- $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$

- $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$

Combine result of recursive call with "last step"

- $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

# Adding Empty Transitions

- Define the set  $\varepsilon\text{-REACHABLE}(q)$  ...
  - ... to be all states reachable from  $q$  via zero or more empty transitions

(Defined recursively)

- Base case:  $q \in \varepsilon\text{-REACHABLE}(q)$

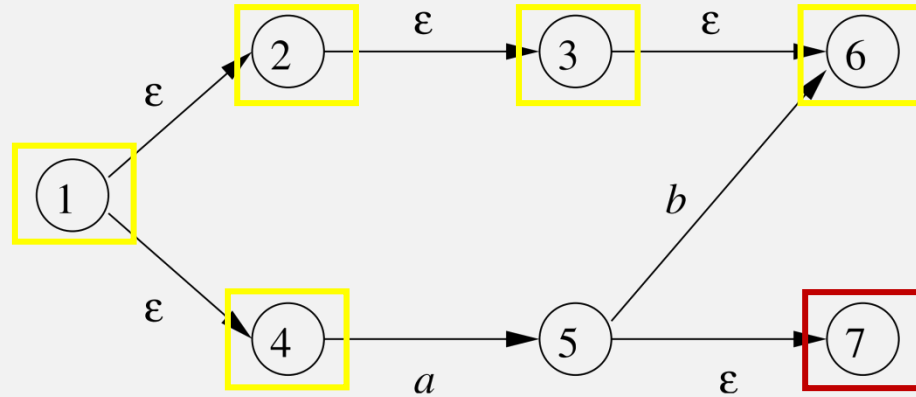
- Recursive case:

A state is in the reachable set if ...

$$\varepsilon\text{-REACHABLE}(q) = \{r \mid p \in \varepsilon\text{-REACHABLE}(q) \text{ and } r \in \delta(p, \varepsilon)\}$$

... there is an empty transition to it from another state in the reachable set

# $\epsilon$ -REACHABLE Example



$$\epsilon\text{-REACHABLE}(1) = \{1, 2, 3, 4, 6\}$$

# NFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - states  $qs \subseteq Q$

where  $w' = w_1 \cdots w_{n-1}$

$$\hat{\delta}(q, w') = \{q_1, \dots, q_k\}$$

(Defined recursively)

Base case

$$\hat{\delta}(q, \epsilon) = \epsilon\text{-REACHABLE}(q)$$

Recursive Case

$$\hat{\delta}(q, w'w_n) =$$

$$\bigcup_{i=1}^k \delta(q_i, w_n) = \{r_1, \dots, r_\ell\}$$

# NFA Multi-Step Transition Function

$$\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

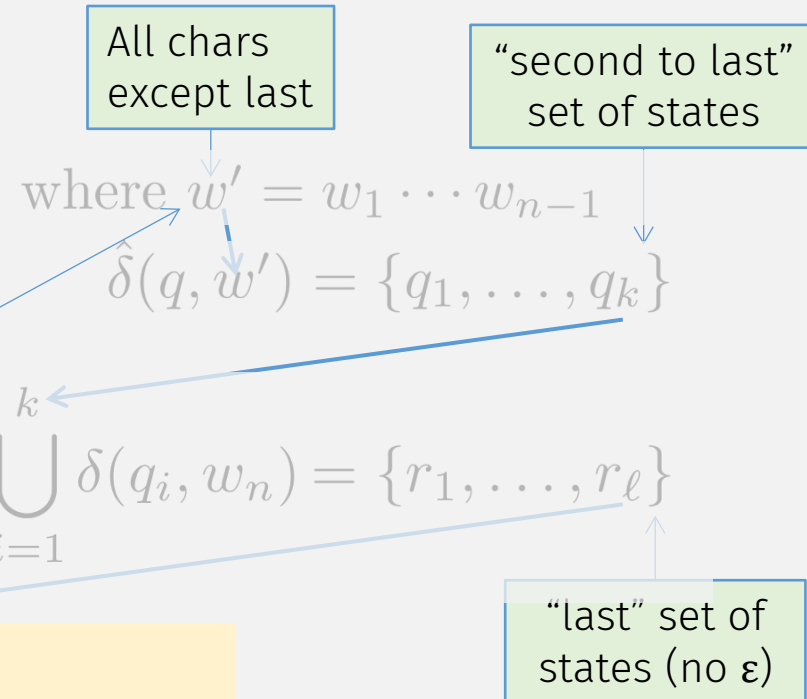
- Domain (inputs):
  - state  $q \in Q$  (doesn't have to be start state)
  - string  $w = w_1 w_2 \cdots w_n$  where  $w_i \in \Sigma$
- Range (output):
  - states  $qs \subseteq Q$

(Defined recursively)

Base case  $\hat{\delta}(q, \epsilon) = \epsilon\text{-REACHABLE}(q)$

Recursive Case

$$\hat{\delta}(q, w'w_n) = \bigcup_{j=1}^{\ell} \epsilon\text{-REACHABLE}(r_j)$$





# Summary: NFA vs DFA Computation

## DFAs

- Can only be in one state
- Transition:
  - Must read 1 char
- Acceptance:
  - If final state is accept state

## NFAs

- Can be in multiple states
- Transition
  - Has empty transitions
- Acceptance:
  - If one of final states is accept state

*Previously*

**Concatenation:**  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Want to prove:

**THEOREM**

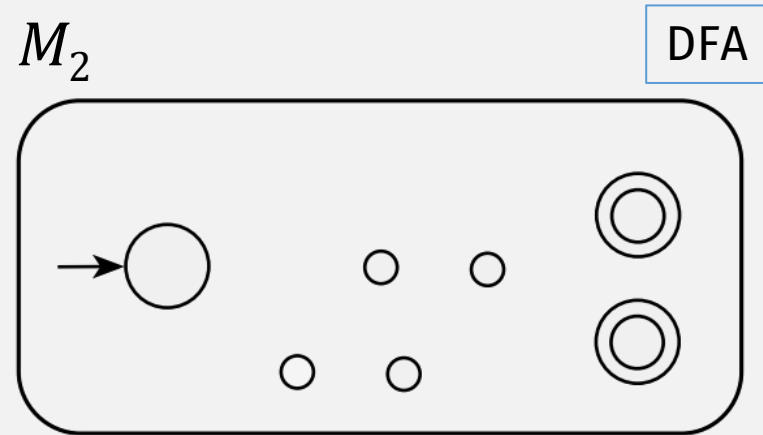
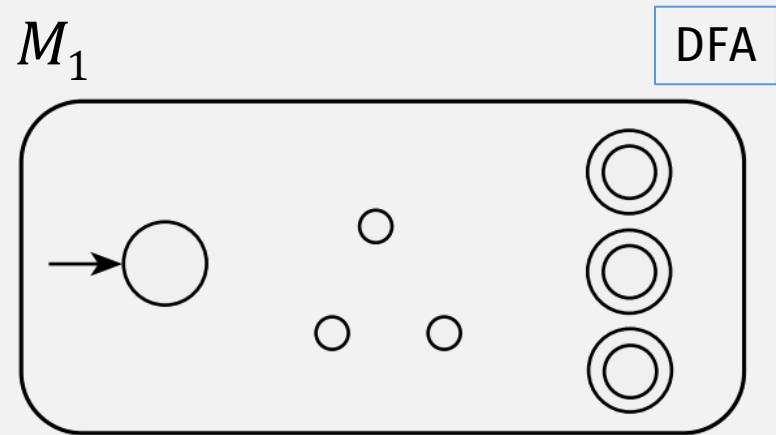
The class of regular languages is closed under the concatenation operation.

In other words, IF  $A_1$  IS-A **Regular** Language AND  $A_2$  IS-A **Regular** Language  
THEN  $A_1 \circ A_2$  IS-A **Regular** Language

*Proof requires:* Constructing new machine

- How does it know when to switch machines?
  - Can only read input once

# Concatentation



Have: recognize  $A_1$  recognize  $A_2$ .

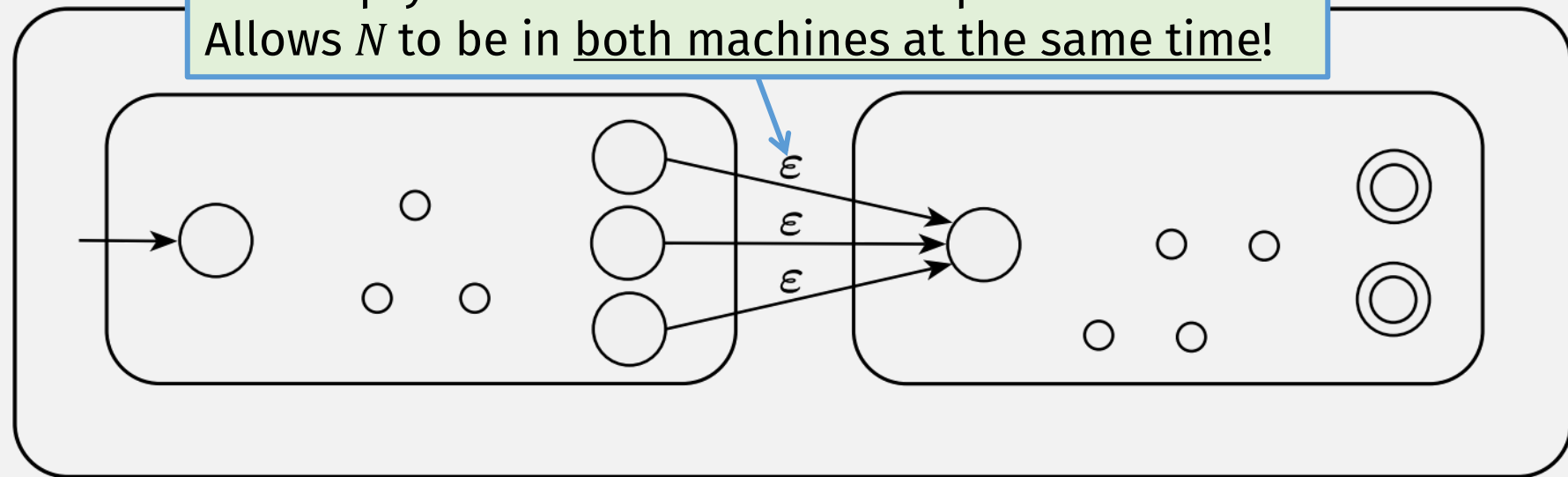
Want: Construction of  $N$  to recognize  $A_1 \circ A_2$

$N$  is an **NFA**! It can:

- Keep checking 1<sup>st</sup> part with  $M_1$  and
- Move to  $M_2$  to check 2<sup>nd</sup> part

$N$

$\epsilon$  = “empty transition” = reads no input  
Allows  $N$  to be in both machines at the same time!



NFA!

Define function  $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2) = N$ , where

let  $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let  $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

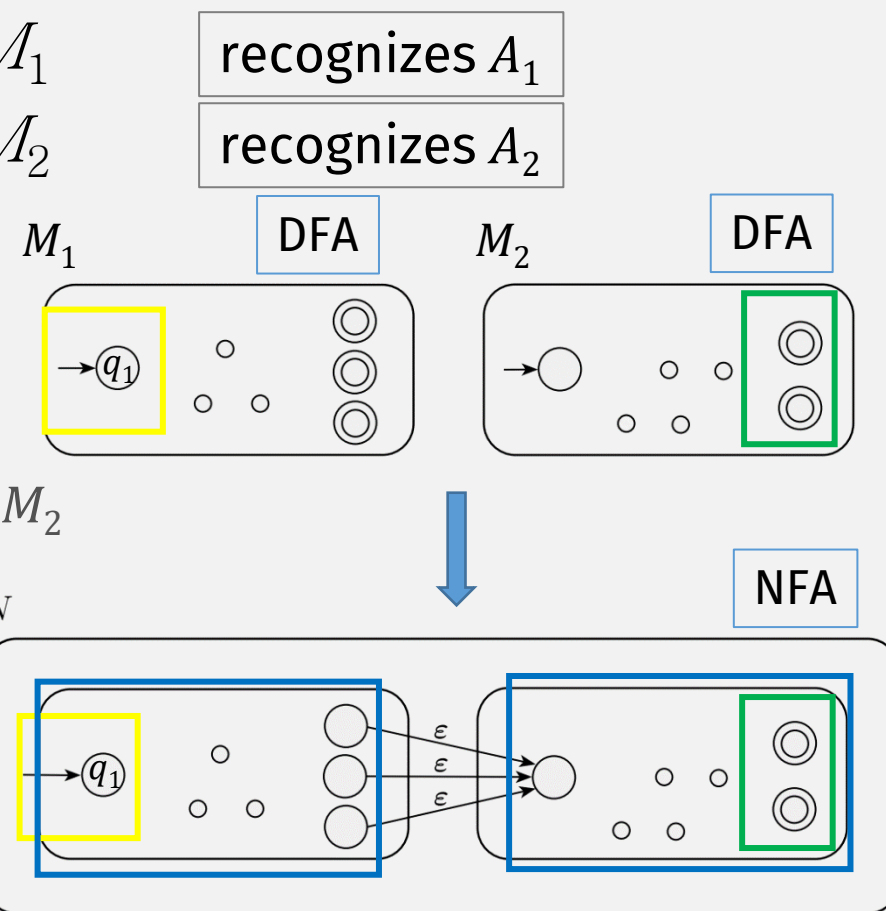
(Output)  $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$  where

$$Q = Q_1 \cup Q_2$$

The state  $q_1$  is the same as the start state of  $M_1$

The accept states  $F_2$  are the same as the accept states of  $M_2$

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$  where: for any  $q \in Q$  and any  $a \in \Sigma_\epsilon$ ,



Define function  $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2) = N$ , where

let  $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let  $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

(Output)  $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$  where

$$Q = Q_1 \cup Q_2$$

The state  $q_1$  is the same as the start state of  $M_1$

The accept states  $F_2$  are the same as the accept states of  $M_2$

$\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$  where: for any  $q \in Q$  and any  $a \in \Sigma_\epsilon$ ,

$$\delta(q, a) = \begin{cases} \{\delta_1(q, a)\} & q \in Q_1 \text{ and } q \notin F_1 \\ \{\delta_1(q, a)\} & q \in F_1 \text{ and } a \neq \epsilon \\ ? & q \in F_1 \text{ and } a = \epsilon \\ \{\delta_2(q, a)\} & q \in Q_2 \end{cases}$$

And:  $\delta(q, \epsilon) = \emptyset$ , for  $q \in Q, q \notin F_1$

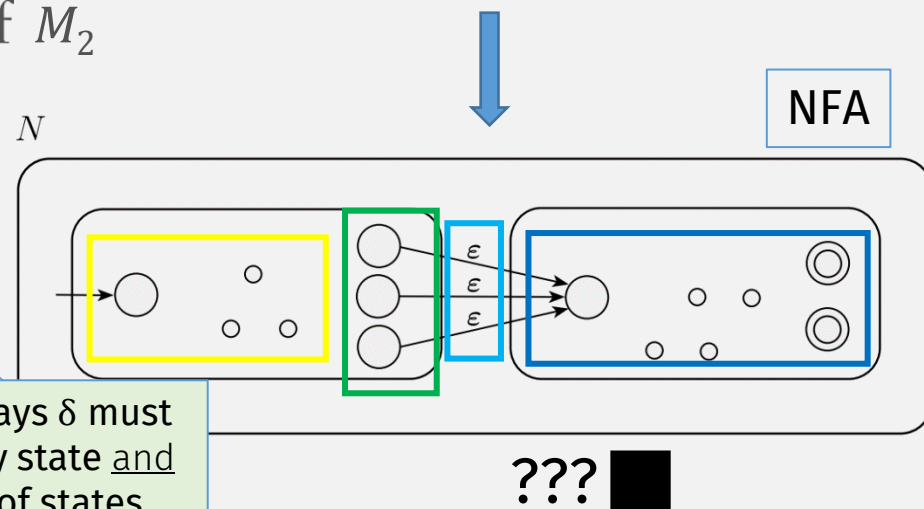
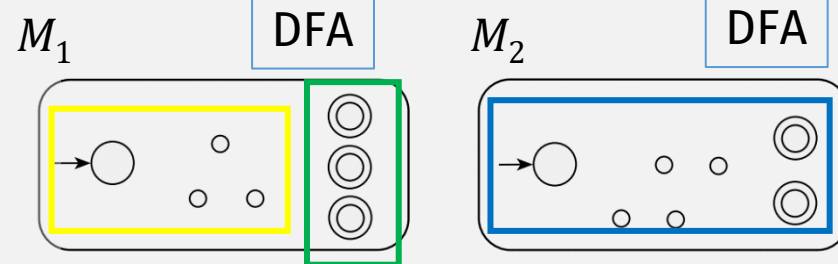
("piecewise"  
function  
definition)

IF  $A_1$  IS-A **Regular Language**  
AND  
 $A_2$  IS-A **Regular Language**  
THEN  $A_1 \circ A_2$  IS-A **Regular Language**

So, is this true?

recognizes  $A_1$

recognizes  $A_2$



NFA def says  $\delta$  must  
map every state and  
 $\epsilon$  to set of states

Previously

Regular Langs closed under Union?, i.e.,

IF  $A_1$  IS-A **Regular** Language AND  
 $A_2$  IS-A **Regular** Language  
THEN  $A_1 \cup A_2$  IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of  $A_1$  IS-A **Regular** Language AND  $A_2$  IS-A **Regular** Language):

### Statements

1.  $A_1$  IS-A **Regular** Language
2.  $A_2$  IS-A **Regular** Language
3.  $M_1$  IS-A **DFA Machine** AND  
 $M_1$  **recognizes**  $A_1$
4.  $M_2$  IS-A **DFA Machine** AND  
 $M_2$  **recognizes**  $A_2$
5. let  $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$   
IS-A **DFA Machine**
6.  $M$  **recognizes**  $A_1 \cup A_2$
7.  $A_1 \cup A_2$  IS-A **Regular** Language

### Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro  
TO Proof<sub>1</sub>
4. APPLY DefRegLangCoro  
TO Proof<sub>2</sub>
5. APPLY  $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$   
TO  ~~$M_1$~~  : **FIRST** Proof<sub>1</sub>, **FIRST** Proof<sub>2</sub>
6. BY ExTable<sub>1</sub> **COMPARING**  $M, A_1 \cup A_2$
7. APPLY DefRegLang  
TO mk-AND-proof Proof<sub>5</sub>, Proof<sub>6</sub>

Regular Langs closed under Concat?, i.e.,

IF  $A_1$  IS-A **Regular** Language AND  
 $A_2$  IS-A **Regular** Language  
THEN  $A_1 \circ A_2$  IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of  $A_1$  IS-A **Regular** Language AND  $A_2$  IS-A **Regular** Language):

### Statements

1.  $A_1$  IS-A **Regular** Language
2.  $A_2$  IS-A **Regular** Language
3.  $M_1$  IS-A **DFA** Machine AND  
 $M_1$  **recognizes**  $A_1$
4.  $M_2$  IS-A **DFA** Machine AND  
 $M_2$  **recognizes**  $A_2$
5. let  $M = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$   
IS-A **NFA** Machine
6.  $M$  **recognizes**  $A_1 \circ A_2$
7.  $A_1 \circ A_2$  IS-A **Regular** Language

### Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro  
TO Proof<sub>1</sub>
4. APPLY DefRegLangCoro  
TO Proof<sub>2</sub>
5. APPLY  $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$   
TO FIRST Proof<sub>1</sub>, FIRST Proof<sub>2</sub>
6. BY ExTable<sub>1</sub> **COMPARING**  $M, A_1 \circ A_2$
7. APPLY DefRegLang  
TO mk-AND-proof Proof<sub>5</sub>, Proof<sub>6</sub>

Regular Langs closed under Concat?, i.e.,

IF  $A_1$  IS-A **Regular** Language AND  
 $A_2$  IS-A **Regular** Language  
THEN  $A_1 \circ A_2$  IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of  $A_1$  IS-A **Regular** Language AND  $A_2$  IS-A **Regular** Language):

### Statements

1.  $A_1$  IS-A **Regular** Language
2.  $A_2$  IS-A **Regular** Language
3.  $M_1$  IS-A **DFA** Machine AND  $M_1$  recognizes  $A_1$  DefRegLang
4. IF  $M$  IS-A **DFA** Machine AND  $M$  recognizes  $L$   
THEN  $L$  IS-A **Regular** Language
5. let  $M = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$   
IS-A **NFA** Machine Doesn't match!
6.  $M$  recognizes  $A_1 \circ A_2$
7.  $A_1 \circ A_2$  IS-A **Regular** Language

### Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro TO Proof<sub>1</sub>
4. APPLY DefRegLangCoro TO Proof<sub>2</sub>
5. APPLY  $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$   
TO FIRST Proof<sub>1</sub>, FIRST Proof<sub>2</sub>
6. BY ExTable<sub>1</sub> COMPARING  $M, A_1 \circ A_2$
7. APPLY DefRegLang TO mk-AND-proof Proof<sub>5</sub>, Proof<sub>6</sub>



IF  $M$  IS-A **DFA** Machine AND  $M$  **recognizes**  $L$   
THEN  $L$  IS-A **Regular** Language

# A DFA's Language

- For **DFA**  $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$  where
- $M$  **accepts**  $w$  if  $\hat{\delta}(q_0, w) \in F$
- $M$  **recognizes** language  $\{w \mid M \text{ accepts } w\}$

Definition: A DFA's language is a **regular language**

# An NFA's Language?

IF  $N$  IS-A **NFA** Machine AND  $N$  **recognizes**  $L$   
THEN  $L$  IS-A ??? Language

- For **NFA**  $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$  where
  - Intersection ...
  - ... with accept states ...
- $N$  **accepts**  $w$  if  $\hat{\delta}(q_0, w) \cap F \neq \emptyset$ 
  - ... is not empty set
  - i.e., accept if final states contains at least one accept state
- Language of  $N = \text{LangOf}(N) = \left\{ w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}$

Q: What kind of languages do NFAs recognize?

# Concatenation Closed for Reg Langs?

- Combining DFAs to recognize concatenation of languages ...  
... produces an NFA
- So to prove concatenation is closed ...  
... we must prove that NFAs also recognize regular languages.

Specifically, we must show:

**NFAs recognize Regular languages too!**

# “IF AND ONLY IF” Statements

$$X \Leftrightarrow Y = \text{“}X \text{ IF AND ONLY IF } Y\text{”} = X \text{ IFF } Y = X \Leftrightarrow Y$$

Represents two statements:

Acceptable notation in this course

1.  $\Rightarrow$  IF  $X$  THEN  $Y$ 
  - “forward” direction
  
2.  $\Leftarrow$  IF  $Y$  THEN  $X$ 
  - “reverse” direction

# How to Prove an “IFF” Statement

$$X \Leftrightarrow Y = \text{“}X \text{ IF AND ONLY IF } Y\text{”} = X \text{ IFF } Y = X \Leftrightarrow Y$$

Proof has two (IF-THEN proof) parts:

1.  $\Rightarrow$  IF  $X$  THEN  $Y$ 
  - “**forward**” direction
  - assume  $X$ , then use it to prove  $Y$
2.  $\Leftarrow$  IF  $Y$  THEN  $X$ 
  - “**reverse**” direction
  - assume  $Y$ , then use it to prove  $X$

# Proving NFAs Recognize Regular Langs

$L$  IS-A **Regular** Language

RegIFFNFA

IF AND ONLY IF

(some)  $N$  IS-A NFA Machine AND  $N$  **recognizes**  $L$

Need 2 proofs:

$\Rightarrow$  IF  $L$  IS-A **Regular** Language THEN  $N$  IS-A NFA Machine AND  $N$  **recognizes**  $L$

Constructor?

$\Leftarrow$  IF  $N$  IS-A NFA Machine AND  $N$  **recognizes**  $L$  THEN  $L$  IS-A **Regular** Language

$\Rightarrow$  IF  $L$  IS-A **Regular** Language  
THEN  $N$  IS-A NFA Machine  
AND  $N$  **recognizes**  $L$

mk-IFTHEN-proof (with ASSUMPTION = proof of  $L$  IS-A **Regular** Language):

**Statements**

1.  $M$  IS-A DFA Machine AND  
 $M$  **recognizes**  $L$
2.  $N$   
IS-A NFA Machine
3.  $N$  **recognizes**  $L$
4.  $N$  IS-A NFA Machine AND  
 $N$  **recognizes**  $L$

**Justifications**

1. **APPLY** DefRegLangCoro  
**TO** ASSUMPTION

DefRegLangCoro

IF  $L$  IS-A **Regular** Language  
THEN  $M$  IS-A **DFA** Machine AND  $M$  **recognizes**  $L$

3. **BY** ExTab? **COMPARING**  $N, L$
4. **mk-AND-proof** Proof<sub>2?</sub> Proof<sub>3?</sub>

$\Rightarrow$  IF  $L$  IS-A **Regular** Language  
THEN  $N$  IS-A NFA Machine  
AND  $N$  **recognizes**  $L$

mk-IFTHEN-proof (with ASSUMPTION = proof of  $L$  IS-A **Regular** Language):

**Statements**

1.  $M$  IS-A DFA Machine AND  
 $M$  **recognizes**  $L$
2. let  $N$   
IS-A NFA Machine See HW4!
3.  $N$  **recognizes**  $L$
4.  $N$  IS-A NFA Machine AND  
 $N$  **recognizes**  $L$

**Justifications**

1. APPLY DefRegLangCoro  
TO ASSUMPTION
2. APPLY  $\text{CONVERT}_{\text{DFA-NFA}} : \text{DFA} \rightarrow \text{NFA}$   
TO FIRST Proof<sub>1</sub>
3. BY ExTab? **COMPARING**  $N, L$
4. mk-AND-proof Proof<sub>2</sub>? Proof<sub>3</sub>?



# “Proving” Machine Recognizes Language

Let:

$$\text{NFA } N = \text{CONVERT}_{\text{DFA-NFA}} M$$

| let $s =$     | $s \in L$ | run $M$ on $s$                         | run $N$ on $s$ ? | run $N$ on $s$ Justification |
|---------------|-----------|----------------------------------------|------------------|------------------------------|
| $w \in L$     | Yes       | $q_0, \dots, q_n$ where $q_n \in F$    | Accept           | See HW4!                     |
| $w' \notin L$ | No        | $q_0, \dots, q_m$ where $q_m \notin F$ | Reject           |                              |
| ...           |           |                                        |                  |                              |

Do we know concrete  $N$  and  $L$ ?

# Proving NFAs Recognize Regular Langs

$L$  IS-A **Regular** Language

RegIFFNFA

IF AND ONLY IF

(some)  $N$  IS-A **NFA Machine** AND  $N$  **recognizes**  $L$

Need 2 proofs:

☑  $\Rightarrow$  IF  $L$  IS-A **Regular** Language THEN  $N$  IS-A **NFA Machine** AND  $N$  **recognizes**  $L$

$\Leftarrow$  IF  $N$  IS-A **NFA Machine** AND  $N$  **recognizes**  $L$  THEN  $L$  IS-A **Regular** Language

Constructor?